

Restaurant Sales Dashboard (frontend-v1)

This is a responsive restaurant sales dashboard built with plain HTML, CSS, and modern JavaScript. The frontend consumes sales data from an HTTP endpoint, draws four charts with Chart.js, and supports Spanish/English UI with light/dark mode.

For the full project annex in English, see [ANEXO_ESTADISTICAS_INTELIGENTES_EN.md](#).

Technologies

- HTML5 / CSS3
- JavaScript ES Modules
- Chart.js
- Bootstrap 5
- Font Awesome
- Google Fonts

Local development

No `npm` or bundler is required.

1. Open `index.html` for the Spanish version or `en.html` for the English version directly in your browser.
2. Click the **Load data** button to fetch sales and render the charts.
3. Use the theme button 🌙/☀️ to switch between dark and light mode; the choice is saved in `localStorage`.

If your browser blocks local `file://` module loading, use the VS Code Live Server extension or another local web server.

Usage

- `index.html`: Spanish dashboard.
- `en.html`: English dashboard.
- `error.html`: static error page that can be used in an S3 deployment.
- `main.js`: main logic for API access, charts, language, and theme.
- `translations.js`: UI text definitions for `es` and `en`.
- `utils.js`: shared utility functions for theme, formatting, colors, and legends.
- `style.css`: responsive styles, dark mode, and dashboard layout.

Features

- Four charts with Chart.js:
 - Revenue by category (pie).
 - Orders by hour (doughnut).
 - Best-selling items (pie).
 - Daily revenue (bar).

- Compact legends with up to 5 labels and a "+N more" summary.
- Manual data loading with a loading indicator.
- Spanish and English UI.
- Persistent dark/light theme stored in `localStorage`.

API and data format

The frontend loads data from this endpoint:

```
GET https://9nccdykio2.execute-api.eu-north-1.amazonaws.com/prod/sales
```

The expected response contains a `salesRecords` array with sales objects:

```
{
  "salesRecords": [
    {
      "date": "2025-01-15",
      "totalRevenue": 1200.5,
      "revenueByCategory": {
        "Entrantes": 300,
        "Postres": 150
      },
      "ordersByHour": {
        "12": 5,
        "13": 8
      },
      "itemsSold": [
        { "itemId": "PAELLA-01", "qty": 3 }
      ]
    }
  ]
}
```

The code aggregates these records to build category, hourly, item, and daily charts.

Deployment

This project is designed to work as a static site hosted in an S3 bucket.

- Bucket: `restaurant-sales-dashboard`
- Region: `eu-north-1`
- URL: `https://restaurant-sales-dashboard.s3.eu-north-1.amazonaws.com`

No build step or bundler is required: upload the files as-is.

Notes

- `main.js` detects the language from the `data-page-lang` attribute on the `html` element.
- If the dashboard does not load data, verify the endpoint and CORS configuration.

Git repository

<https://github.com/MSTVLC/frontend-v1.git>

Contact

Restaurant Estrella — support@mysite.com · +34 969 636 773

Regenerate PDFs

If you need to regenerate the PDFs from Markdown without extra headers or footers, use the script:

```
python3 scripts/generate_pdfs_reportlab.py
```